



CHRIST

(DEEMED TO BE UNIVERSITY)

BANGALORE • INDIA

DEPARTMENT OF COMPUTER SCIENCE

LnT CIA 1 Theory and Practical Questions

Submitted by

Lakshmi Prakash -2460396

5 BTCS A

B. Tech – Computer Science and Engineering

School of Engineering and Technology,

CHRIST (Deemed to be University),

Kumbalagodu, Bengaluru-560 074

September 2026

Code and Outputs:

Github Link : <https://github.com/laksh-me4/CIA2---LnT>

Task 1 :

```
JS Task1.js
1 // Task 1: Basic Node.js application
2
3 console.log("Countdown App Ready");
```

```
Problems Output Terminal ... powershell + - [ ] [ ] ...
● PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> node Task1.js
Countdown App Ready
❖ PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> 
```

Task 2:

```
JS Task2.js > ...
1 // Task 2: Understanding Node.js Architecture
2
3 console.log("Starting timer...");
4
5 setTimeout(() => {
6   console.log("Timer finished!");
7 }, 3000);
8
9 console.log("Node.js is still running...");
```

```
Problems Output Terminal ... powershell + - [ ] [ ] ...
● PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> node Task2.js
Starting timer...
Node.js is still running...
Timer finished!
❖ PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> 
```

Task 3:

```
JS Task3.js > ...
1 // Task 3: Basic countdown using setInterval
2 let seconds = 5;
3
4 const timer = setInterval(() => {
5   console.log("Time remaining:", seconds);
6   seconds--;
7
8   if (seconds < 0) {
9     clearInterval(timer);
10    console.log("Countdown finished!");
11  }
12 }, 1000);
```

```
Problems Output Debug Console Terminal Ports powershell + - [ ] [X]
● PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> node Task3.js
Time remaining: 5
Time remaining: 4
Time remaining: 3
Time remaining: 2
Time remaining: 1
Time remaining: 0
Countdown finished!
❖ PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> |
```

Task 5:

```
JS Task5.js > ...
1 // Task 5: Command Line Arguments and Terminal Input
2
3 const seconds = parseInt(process.argv[2]);
4
5 if (isNaN(seconds) || seconds <= 0) {
6     console.log("Please provide a valid positive number.");
7     process.exit(1);
8 }
9
10 console.log(`Countdown started for ${seconds} seconds.`);
11 console.log('Type "cancel" and press Enter to stop the countdown.');
```

```
12
13 let remaining = seconds;
14
15 const timer = setInterval(() => {
16     console.log(`Time remaining: ${remaining} seconds`);
17
18     remaining--;
19
20     if (remaining < 0) {
21         clearInterval(timer);
22         console.log("Countdown finished!");
23         process.stdin.pause();
24     }
25 }, 1000);
26
27 // Read input from terminal
28 process.stdin.setEncoding("utf8");
29
30 process.stdin.on("data", (input) => {
31     const command = input.trim().toLowerCase();
32
33     if (command === "cancel") {
34         clearInterval(timer);
35         console.log("Countdown cancelled!");
36         process.stdin.pause();
37     }
38 });
39
40 process.stdin.resume();
```

```

● PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> node Task5.js 10
Countdown started for 10 seconds.
Type "cancel" and press Enter to stop the countdown.
Time remaining: 10 seconds
Time remaining: 9 seconds
Time remaining: 8 seconds
Time remaining: 7 seconds
cancelTime remaining: 6 seconds
      Time remaining: 5 seconds
      1
Countdown cancelled!
❖ PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT>

```

Task 7:

```

JS Task7.js > ...
1  // Task 7: Debugging Node.js Program
2
3  let seconds = 5;
4
5  const timer = setInterval(() => {
6    console.log("Time remaining:", seconds);
7
8    seconds--;
9
10   if (seconds === 0) {
11     clearInterval(timer);
12     console.log("Countdown finished!");
13   }
14 }, 1000);
15
16 /*
17 Debugging:
18 I placed a breakpoint inside the setInterval callback
19 and checked the value of seconds after each iteration.
20 The comparison operator was incorrect.
21 I changed the condition so that the timer stops
22 when seconds reaches zero.
23 */

```

```

● PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> node Task7.js
Time remaining: 5
Time remaining: 4
Time remaining: 3
Time remaining: 2
Time remaining: 1
Countdown finished!
❖ PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT>

```

Task 8:

```
JS Task8.js > ...
1 // Task 8: Callback-based asynchronous function
2
3 function checkTimeLeftCallback(seconds, callback) {
4
5     // Simulate an asynchronous delay
6     setTimeout(() => {
7
8         const remainingTime = seconds;
9
10        callback(null, remainingTime);
11
12    }, 2000);
13 }
14
15 // Calling the function
16 checkTimeLeftCallback(5, (error, remaining) => {
17
18     if (error) {
19         console.log("Error:", error);
20         return;
21     }
22
23     console.log("Remaining time:", remaining, "seconds");
24 });
```

```
PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> node Task8.js
Remaining time: 5 seconds
PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT>
```

Task 10:

```
JS Task10.js > ...
1 // Task 10: Promise-based countdown function
2
3 function checkTimeLeftPromise(seconds) {
4
5     return new Promise((resolve, reject) => {
6
7         // Reject invalid duration
8         if (seconds < 0) {
9             reject(new Error("Duration cannot be negative."));
10            return;
11        }
12
13        // Simulate asynchronous operation
14        setTimeout(() => {
15            resolve(`Remaining time: ${seconds} seconds`);
16        }, 2000);
17    });
18 }
19
20 // Promise chaining
21 checkTimeLeftPromise(5)
22 .then((result) => {
23     console.log(result);
24 })
25 .catch((error) => {
26     console.log("Error:", error.message);
27 });
```

```

Remaining time: 5 seconds
PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> node Task10.js
Remaining time: 5 seconds
❖ PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT>

```

Task 11:

```

JS Task11-2.js > ...
1  // Task 11: Async/Await and Error Handling
2
3  function checkTimeLeftPromise(seconds) {
4
5      return new Promise((resolve, reject) => {
6
7          if (seconds < 0) {
8              reject(new Error("Duration cannot be negative."));
9              return;
10         }
11
12         setTimeout(() => {
13             resolve(`Remaining time: ${seconds} seconds`);
14         }, 1000);
15     });
16 }
17
18 async function runCountdownAsync(seconds) {
19
20     try {
21
22         const result = await checkTimeLeftPromise(seconds);
23
24         console.log(result);
25
26     } catch (error) {
27
28         console.log("Caught error:", error.message);
29     }
30 }
31
32 // Invalid negative duration
33 runCountdownAsync(5);

```

```

❖ PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> node Task11.js
Caught error: Duration cannot be negative.
❖ PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> node Task11-2.js
Remaining time: 5 seconds
❖ PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT>

```

Task 12:

```
JS Task12.js > runCountdownAsync
1 // Task 12: Call Stack and Callback Queue
2
3 function checkTimeLeftPromise(seconds) {
4
5     return new Promise((resolve) => {
6
7         setTimeout(() => {
8             resolve(`Time remaining: ${seconds} seconds`);
9         }, 1000);
10
11     });
12 }
13
14 async function runCountdownAsync(seconds) {
15
16     try {
17
18         console.log("1. Async function started");
19
20         const result = await checkTimeLeftPromise(seconds);
21
22         console.log("3. Promise resolved:", result);
23
24     } catch (error) {
25
26         console.log("Error:", error.message);
27
28     }
29 }
30
31 runCountdownAsync(5);
32
33 function recursiveCountdown(n) {
34
35     console.log(n);
36
37     if (n === 0) {
38         return;
39     }
40
41     recursiveCountdown(n - 1);
42 }
43
```

```
PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT> node Task12.js
1. Async function started
3. Promise resolved: Time remaining: 5 seconds
PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT>
```

Task 15:

```
countdown-app-TASK15 > JS countdownModule.js > ...
1 // Task 15: Local Node.js Module
2
3 function startCountdown(seconds) {
4
5     if (seconds <= 0) {
6         console.log("Please provide a positive duration.");
7         return;
8     }
9
10    console.log(`Countdown started for ${seconds} seconds.`);
11
12    let remaining = seconds;
13
14    const timer = setInterval(() => {
15
16        console.log("Time remaining:", remaining);
17
18        remaining--;
19
20        if (remaining === 0) {
21            clearInterval(timer);
22            console.log("Countdown finished!");
23        }
24    }, 1000);
25
26
27 }
28
29
30 // Export the function
31 module.exports = {
32     startCountdown
33 };
```

```
countdown-app-TASK15 > JS countdown.js > ...
1 // Task 15: Importing a Local Module
2
3 // Import countdownModule.js using require()
4 const countdownModule = require("./countdownModule");
5
6 // Get seconds from command line
7 const seconds = parseInt(process.argv[2]) || 5;
8
9 // Start countdown
10 countdownModule.startCountdown(seconds);
```

```
● PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT\countdown-app-TASK15> node countdown.js
Countdown started for 5 seconds.
Time remaining: 5
Time remaining: 4
Time remaining: 3
Time remaining: 2
Time remaining: 1
Countdown finished!
❖ PS C:\Users\USER\OneDrive\Desktop\CIA2 - LNT\countdown-app-TASK15> |
```


Ans 1.

If app.js is not present in the current folder, Node.js will report Error: Cannot find module & show code: MODULE_NOT_FOUND. The developer should either use cd to move to the folder containing app.js or provide the correct file path.

Ans 2.

The V8 engine is responsible for executing JavaScript code by compiling it into machine code, it also provides the event loop & handles asynchronous operations such as network requests, file operations & timers.

Ans 3

The Node.js JS API documentation provides information about file system functions including their syntax, parameters, return values, error & whether they are synchronous / asynchronous.

Ans 5

For the command node app.js hello world, process.argv contains the Node.js executable path, the path of app.js and the two supplied arguments. Therefore its structure is: [node-path, app.js-path, "hello", "world"]. The first 2 values are system dependent paths while hello & world are exactly the command line arguments provided by user.

Ans 7.

The problem is that getUser() returns a promise but the code tries to access user.name before the promise has been resolved. The fix is to wait for the promise using await inside an async function. Alternatively .then() can be used to access the resolved user object.

Ans 8.

The function should use `fs.readFile()` & follow the error-first callback convention where the first argument represents an error & the second contains the result.

```
const fs = require('fs');  
function readfileCallback(path, callback) {  
  fs.readFile(path, 'utf8', (err, data) => {  
    if (err) {  
      callback(err, null);  
    }  
    else {  
      callback(null, data);  
    }  
  });  
}
```

Ans 10.

A Promise based delay function can be written using `setTimeout()` as follows:

```
function delay(ms) {  
  return new Promise((resolve) => {  
    setTimeout(resolve, ms);  
  });  
}
```

Ans 11.

`Promise.reject('err')` creates a rejected Promise. The `await` causes the rejection to be caught by the catch block, where `console.log()` prints the error. Output is:
caught : err

Ans 12.

When `a()` is called it first prints `a` & then calls `b()`. Function `b()` prints `b` and returns to `a()`. After `a()` finishes the next statement `console.log('c')` executes. Output is
a
b
c

Ans 13.

In `math.js`, the function can be exported using `module.exports`:

```
function add(a, b) {  
  return a + b;  
}  
module.exports = add;
```

In `app.js`, it can be imported using `require()` & called as follows:

```
const add = require('./math');  
console.log(add(5, 3));
```